

FFmpeg 转码 & Mac 打包部署文档

桌面宠物 · 团子陪伴 v7.0.0

版本: V1.0

日期: 2026 年 8 月

文档类型: 技术部署文档

目录

一、概述	4
适用范围	4
二、环境准备	4
2.1 系统要求	4
2.2 安装 FFmpeg	4
方式一：Homebrew 安装（推荐）	4
方式二：官网下载	5
2.3 Python 依赖安装	5
三、视频转 GIF 转码流程	6
3.1 素材目录结构	6
3.2 MP4 → 透明 GIF 转换	6
核心函数：black_to_transparent	7
运行转换脚本	7
3.3 GIF 优化压缩	8
3.4 批量转换脚本详解	8
CONVERT_MAP（需要 MP4 → GIF 转码）	8
COPY_MAP（已有 GIF 直接复制）	9
四、视频编码检测与自动转码	9
4.1 VideoTranscoder 类	9
4.2 H.264 编码转码命令	10
标准转码（推荐）	10
高质量转码	10
批量转码脚本	10
五、绿幕抠像与透明通道处理	11
5.1 抠像配置参数	11
5.2 黑底转透明原理	11
六、Mac .app 打包部署	12
6.1 打包脚本架构设计	12
打包执行流程	12

6.2 .app 目录结构规范	12
6.3 启动脚本核心逻辑	13
① 环境变量与编码设置	13
② 资源目录动态解析	13
③ 可写数据目录	14
④ 虚拟环境自举	14
⑤ Qt 插件路径动态检测（关键！）	15
⑥ 启动应用与日志	16
6.4 Info.plist 配置	16
6.5 首次运行自动初始化	17
七、代码签名与分发	18
7.1 Ad-hoc 签名	18
Ad-hoc 签名的局限性	18
（可选）开发者证书签名 + 公证	18
7.2 分发注意事项	19
八、常见问题与排查	19
8.1 启动日志定位	19
8.2 Qt 插件错误修复	20
错误：qt.qpa.plugin Could not find the Qt platform plugin "cocoa"	20
8.3 视频无法播放排查	20
九、附录：完整命令速查表	20

一、概述

本文档详细介绍《团子陪伴》桌面宠物应用的 FFmpeg 视频转码流程与 macOS .app 打包部署全流程。涵盖从原始视频素材处理、透明 GIF 生成、绿幕抠像，到 Python 虚拟环境自举、Qt 插件路径配置、代码签名与分发的完整技术方案。

适用范围

- 视频素材开发人员：MP4 → GIF 转码、透明通道处理、素材优化
- 桌面端开发人员：PyQt5 应用打包、Qt 插件配置、启动脚本编写
- 运维/测试人员：.app 部署、签名验证、问题排查

二、环境准备

2.1 系统要求

项目	要求
操作系统	macOS 11.0 (Big Sur) 及以上
Python	3.8 ~ 3.12 (推荐 3.10/3.11)
FFmpeg	4.0 及以上 (含 ffprobe)
PyQt5	>= 5.15.0
磁盘空间	至少 500MB (含依赖安装)

2.2 安装 FFmpeg

方式一：Homebrew 安装 (推荐)

```
# 安装 Homebrew (如未安装)

/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 安装 FFmpeg

brew install ffmpeg

# 验证安装

ffmpeg -version

ffprobe -version
```

方式二：官网下载

访问 <https://evermeet.cx/ffmpeg/> 下载静态编译版本，解压后将 ffmpeg 和 ffprobe 复制到 /usr/local/bin/

```
# 解压并复制
sudo cp ffmpeg /usr/local/bin/
sudo cp ffprobe /usr/local/bin/
sudo chmod +x /usr/local/bin/ffmpeg /usr/local/bin/ffprobe
```

验证

```
ffmpeg -version | head -3
```

【注意】 macOS 首次运行可能提示「无法打开」，请在【系统设置 → 隐私与安全性】中点击「仍要打开」。

2.3 Python 依赖安装

```
# 进入项目目录
cd "/Users/suyue/Desktop/材料/desktop_pet 5"

# 创建虚拟环境
python3 -m venv venv
source venv/bin/activate

# 安装基础依赖
pip install -r requirements.txt

# 安装转码工具依赖（用于 convert_gifs.py / optimize_gifs.py）
pip install imageio imageio-ffmpeg pillow numpy

# 验证 PyQt5
python -c "import PyQt5; print('PyQt5:', PyQt5.QtCore.QT_VERSION_STR)"
```

三、视频转 GIF 转码流程

3.1 素材目录结构

```
desktop_pet 5/
├── assets/
│   ├── video/      # 原始视频素材 (MP4/MOV/GIF)
│   │   ├── idle.gif    # 待机 (可直接用 GIF)
│   │   ├── happy.gif   # 开心
│   │   ├── lie_back.mp4 # 平躺 (需要转码)
│   │   ├── lie_side.gif # 侧躺
│   │   ├── sleep.gif   # 睡觉
│   │   ├── stretch.mp4 # 伸懒腰 (需要转码)
│   │   ├── eat.mp4     # 喝水 (需要转码)
│   │   └── ...
│   └── pet_gif/      # 输出透明 GIF 目标目录
│       ├── idle.gif
│       ├── happy.gif
│       ├── lie_back.gif
│       ├── rest.gif
│       ├── stretch.gif
│       ├── wave.gif
│       ├── focus.gif
│       └── tuanzi_icon.png
```

项目支持的视频扩展名 (config.VIDEO_EXTENSIONS) :

```
VIDEO_EXTENSIONS = [".mp4", ".mov", ".avi", ".webm", ".mkv", ".m4v", ".gif"]
```

3.2 MP4 → 透明 GIF 转换

使用 convert_gifs.py 脚本将 MP4 视频转换为带透明通道的 GIF。核心处理流程:

步骤 1: 读取 MP4 全部帧 (通过 imageio-ffmpeg)

步骤 2: 按步长抽帧 (目标 24 帧) 控制 GIF 体积

步骤 3: 统一缩放到 200×200 像素

步骤 4: 黑底像素 (RGB 均 < 35) 转为 Alpha=0 透明

步骤 5: 使用 FASTOCTREE 算法量化为 128 色

步骤 6: 保存为循环播放的 GIF (disposal=2 保证透明帧正确)

核心函数: `black_to_transparent`

```
def black_to_transparent(frame_np, threshold=35):  
    """将黑底像素转为透明"""  
    h, w = frame_np.shape[:2]  
    rgba = np.zeros((h, w, 4), dtype=np.uint8)  
    rgba[:, :, :3] = frame_np  
    # RGB 三通道均低于阈值判定为背景  
    is_bg = np.all(frame_np < threshold, axis=2)  
    rgba[:, :, 3] = np.where(is_bg, 0, 255)  
    return rgba
```

【提示】 threshold=35 是经验值, 若抠像不干净 (有黑边) 可适当调高至 45~55; 若误抠掉猫咪深色部分则调低。

运行转换脚本

```
# 激活虚拟环境  
source venv/bin/activate  
  
# 执行转码  
python convert_gifs.py  
    # 预期输出示例  
Converting lie_back.mp4 -> lie_back.gif  
Total frames: 120, step: 5, target: ~24  
Extracted 24 frames  
Saved: 286 KB  
Converting stretch.mp4 -> stretch.gif  
...  
Copied idle.gif (156 KB)  
Copied happy.gif (198 KB)  
...
```

All 7 GIFs ready:

focus.gif: 142 KB

happy.gif: 198 KB

idle.gif: 156 KB

lie_back.gif: 286 KB

rest.gif: 175 KB

stretch.gif: 234 KB

wave.gif: 189 KB

3.3 GIF 优化压缩

使用 optimize_gifs.py 对已有的 GIF 进一步压缩，减少体积:

优化项	参数	说明
目标尺寸	200 × 200	统一尺寸保证 UI 一致性
最大帧数	30 帧	抽帧减少总量
色深	128 色	FASTOCTREE 量化
帧间隔	100ms	平衡流畅度与体积

```
python optimize_gifs.py

# 输出示例
idle.gif: 60 -> 30 frames, 156 KB
wave.gif: 48 -> 24 frames, 189 KB
rest.gif: 55 -> 28 frames, 175 KB
focus.gif: 62 -> 31 frames, 142 KB
```

【提示】 目标 GIF 单文件建议控制在 100~300 KB 以内，7 个文件总体积 < 2MB，避免 .app 过大。

3.4 批量转换脚本详解

CONVERT_MAP (需要 MP4 → GIF 转码)

```
CONVERT_MAP = {
    "lie_back": "lie_back.mp4", # 撒娇躺平
    "stretch": "stretch.mp4", # 伸懒腰
```



```
"happy": "eat.mp4",    # 喝水开心（重命名映射）
}
```

COPY_MAP（已有 GIF 直接复制）

```
COPY_MAP = {
    "idle": "idle.gif",    # 待机
    "wave": "happy.gif",  # 挥手（复用开心素材）
    "rest": "lie_side.gif", # 休憩（复用侧躺素材）
    "focus": "sleep.gif", # 专注（复用睡觉素材）
}
```

【提示】 COPY_MAP 实现「一份源素材，映射多个动作」，减少重复制作。若需独立动画，替换映射的源文件名即可。

四、视频编码检测与自动转码

4.1 VideoTranscoder 类

pet_window.py 中内置 VideoTranscoder 工具类，运行时检测视频编码并可自动转码为兼容格式：

```
class VideoTranscoder:
    """视频编码检查与转码提示"""

    @staticmethod
    def check_codec(video_path):
        """调用 ffprobe 检查视频编码"""
        cmd = ['ffprobe', '-v', 'quiet', '-print_format', 'json',
              '-show_streams', video_path]
        result = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
        # 返回 codec_name / width / height / duration
        ...

    @staticmethod
    def transcode_to_h264(input_path, output_path):
        """转码为 H.264 + AAC（macOS 兼容性最好的格式）"""
```

```

cmd = [
    'ffmpeg', '-i', input_path,
    '-c:v', 'libx264', # 视频编码 H.264
    '-preset', 'fast', # 编码速度预设
    '-crf', '23',      # 画质因子 (18~28, 越小越清晰)
    '-c:a', 'aac',     # 音频编码 AAC
    '-y', output_path
]
subprocess.run(cmd, ...)

```

4.2 H.264 编码转码命令

标准转码 (推荐)

通用转码命令 (画质均衡)

```

ffmpeg -i input.mov -c:v libx264 -preset medium -crf 23 -c:a aac -b:a 128k -pix_fmt
yuv420p -movflags +faststart output.mp4

```

高质量转码

高画质 (体积约 2 倍)

```

ffmpeg -i input.mov -c:v libx264 -preset slow -crf 18 -c:a aac -b:a 192k -pix_fmt
yuv420p output.mp4

```

批量转码脚本

```

#!/bin/bash

# 批量将 assets/video 下所有非 H.264 视频转码
cd "/Users/suyue/Desktop/材料/desktop_pet 5/assets/video"
mkdir -p converted

for f in *.mov *.avi *.mkv; do
    [ -f "$f" ] || continue
    codec=$(ffprobe -v quiet -select_streams v:0 -show_entries stream=codec_name -of
csv=p=0 "$f")
    if [ "$codec" != "h264" ]; then
        echo "转码: $f (当前编码: $codec)"
    fi
done

```

```
ffmpeg -i "$f" -c:v libx264 -preset fast -crf 23 -c:a aac -y "converted/${f%.*}.mp4"
else
echo "跳过: $f (已是 H.264)"
cp "$f" "converted/"
fi
done
```

五、绿幕抠像与透明通道处理

5.1 抠像配置参数

config.py 中绿幕抠像全局配置:

```
# ===== 绿幕抠像 =====
CHROMA_KEY_ENABLED = True      # 总开关
CHROMA_KEY_COLOR = (0, 255, 0) # 绿幕颜色 (RGB)
CHROMA_KEY_THRESHOLD = 100     # 容差阈值 (0~442, 越大越宽松)
```

场景	CHROMA_KEY_ENABLED	说明
绿幕拍摄素材	True	按 CHROMA_KEY_COLOR 抠除背景
黑底合成素材	False	使用 convert_gifs.py 中的 black_to_transparent
已带 Alpha 通道	False	直接播放, 无需额外抠像

5.2 黑底转透明原理

convert_gifs.py 中使用 numpy 向量化操作, 对每一帧判断 RGB 三通道是否均低于阈值 (默认 35) 。满足条件的像素设为完全透明 (Alpha=0) , 否则完全不透明 (Alpha=255) 。

阈值选择参考:

阈值	适用场景	注意事项
20~30	纯色黑底、对比度极高	可能误抠深色毛发
30~45	默认推荐, 大部分场景	平衡抠像干净度与边缘保留
45~60	背景偏灰或有噪点	需注意不要抠掉猫咪眼睛/

		深色花纹
--	--	------

六、Mac .app 打包部署

6.1 打包脚本架构设计

build_app.sh 采用「动态路径 + 首次自举」架构设计，核心原则：

路径动态计算：所有资源目录在启动时通过 `dirname` 动态解析，`.app` 可移动到任意位置（`/Applications`、桌面、U 盘等）均可运行。

venv 用户隔离：虚拟环境创建在 `~/Library/Application Support/桌面宠物/venv`，不随 `.app` 包内置，避免代码签名复杂度过高。

首次启动自举：首次双击 `.app` 自动创建 `venv`、安装 `requirements.txt`、检测 `PyQt5` 完整性。

Qt 插件自动探测：通过 Python 动态检测 `PyQt5` 插件目录并设置 `QT_PLUGIN_PATH`，兼容所有 `PyQt5` 安装布局。

日志持久化：启动日志写入 `~/Library/Application Support/桌面宠物/launch.log`，便于排查。

打包执行流程

1. 检查 `python3` 是否存在（3.8+）
2. 清理旧 `dist/` 目录
3. 创建 `.app` 目录结构（`Contents/MacOS + Contents/Resources`）
4. 复制项目文件到 `Resources`（`assets`、`*.py`、`requirements.txt`、`animations.json`）
5. 写入启动脚本 `Contents/MacOS/桌面宠物`（设置可执行权限）
6. 写入 `Info.plist`（版本号、`Bundle ID`、`LSUIElement` 等）
7. `codesign` 进行 `ad-hoc` 签名（`--force --deep --sign -`）
8. 输出使用说明与日志路径

6.2 .app 目录结构规范

桌面宠物.app/	
└── Contents/	
├── Info.plist	# 应用元数据配置
├── MacOS/	
├── 桌面宠物	# 可执行启动脚本（ <code>chmod +x</code> ）
└── Resources/	# 所有资源与 Python 代码

```
├── main.py      # 应用入口
├── config.py    # 全局配置
├── pet_window.py # 主窗口
├── ai_chat.py   # AI 对话模块
├── reminder.py  # 健康提醒
├── tools_panel.py # 工具面板
├── glass_widgets.py # 毛玻璃弹窗
├── check_videos.py # 视频检测工具
├── requirements.txt # pip 依赖清单
├── animations.json # 动画元数据（可选）
├── assets/
│   ├── pet_gif/    # GIF 素材
│   └── video/      # MP4/MOV 视频素材
```

6.3 启动脚本核心逻辑

① 环境变量与编码设置

确保中文路径/文件名不出现编码问题

```
export LANG=en_US.UTF-8
```

```
export LC_ALL=en_US.UTF-8
```

【注意】 使用 `en_US.UTF-8` 而非 `zh_CN.UTF-8` 可避免部分精简系统缺失中文 locale 导致的启动失败。

② 资源目录动态解析

```
SELF_DIR="$(cd "$(dirname "$0")" && pwd)"
```

```
RES_DIR="$(cd "${SELF_DIR}/../Resources" 2>/dev/null && pwd)"
```

备用解析方式（处理中文路径编码问题）

```
if [ -z "${RES_DIR}" ] || [ ! -d "${RES_DIR}" ]; then
```

```
    RES_DIR="$(dirname "$0")/../Resources"
```

```
    RES_DIR="$(cd "${RES_DIR}" 2>/dev/null && pwd)"
```

```
fi
```

最终兜底：弹窗报错

```
if [ -z "${RES_DIR}" ] || [ ! -d "${RES_DIR}" ]; then
    osascript -e 'display alert "启动失败" message "无法定位资源目录"'
    exit 1
fi
```

③ 可写数据目录

```
# macOS 标准可写位置 (.app 包内是只读的)
SUPPORT_DIR="$HOME/Library/Application Support/桌面宠物"
LAUNCH_LOG="${SUPPORT_DIR}/launch.log"
VENV_DIR="${SUPPORT_DIR}/venv"
mkdir -p "${SUPPORT_DIR}"
```

【警告】 严禁写入 .app/Contents 下的任何路径！ macOS Gatekeeper 会拒绝修改签名后的 .app 包内容。

④ 虚拟环境自举

```
# 优先使用已有 venv 中的 Python
if [ -f "${VENV_DIR}/bin/python" ]; then
    PYTHON="${VENV_DIR}/bin/python"
else
    # 查找系统 Python（按版本优先级）
    for p in python3 python3.12 python3.11 python3.10 python3.9 python3.8; do
        command -v "${p}" &>/dev/null && { PYTHON="$(command -v "${p}")"; break; }
    done

    # --copies: 复制解释器到 venv（不是 symlink），.app 移动后不会断
    "${PYTHON}" -m venv --copies "${VENV_DIR}"

    # 安装依赖（失败时不终止启动，尽力而为）
    "${VENV_DIR}/bin/pip" install --quiet -r "${RES_DIR}/requirements.txt"

    # 关键检查：PyQt5 必须可用
```

```
"${VENV_DIR}/bin/python" -c "import PyQt5" ||      "${VENV_DIR}/bin/pip" install
PyQt5
fi
```

⑤ Qt 插件路径动态检测（关键！）

```
# 通过 Python 动态检测 PyQt5 的插件目录，兼容所有安装布局
QT_PLUGIN_DIRS=$( "${PYTHON}" -c "
import os, sys
dirs = []
try:
    import PyQt5
    pyqt5_dir = os.path.dirname(PyQt5.__file__)
    # 尝试 3 种常见布局
    for p in [os.path.join(pyqt5_dir, 'Qt5', 'plugins'),
              os.path.join(pyqt5_dir, 'Qt5', 'plugins', 'platforms'),
              os.path.join(pyqt5_dir, 'plugins')]:
        os.path.isdir(p) and dirs.append(p)
    # 最可靠方式: QLibraryInfo
    from PyQt5.QtCore import QLibraryInfo
    plugins_dir = QLibraryInfo.location(QLibraryInfo.PluginsPath)
    if plugins_dir and os.path.isdir(plugins_dir):
        dirs.append(plugins_dir)
        os.path.isdir(os.path.join(plugins_dir, 'platforms')) and
dirs.append(os.path.join(plugins_dir, 'platforms'))
except: pass
print(':'.join(dirs) if dirs else '')
")

# 设置环境变量
if [ -n "${QT_PLUGIN_DIRS}" ]; then
    export QT_PLUGIN_PATH="${QT_PLUGIN_DIRS}:${QT_PLUGIN_PATH:-}"
```

```
export
QT_QPA_PLATFORM_PLUGIN_PATH="${QT_PLUGIN_DIRS}:${QT_QPA_PLATFORM_PL
UGIN_PATH:-}"
fi
```

【重要】这段代码解决了 90% 的「Could not find the Qt platform plugin cocoa」错误。静态写死路径容易因为 PyQt5 版本/安装方式不同而失效。

⑥ 启动应用与日志

```
# 工作目录切到 Resources（使用相对路径加载 assets）
cd "${RES_DIR}"

# 启动 main.py，所有 stdout/stderr 写入 launch.log
"${PYTHON}" main.py 2>&1 | while IFS= read -r line; do
    log "[app] ${line}"
done

EXIT_CODE=${PIPESTATUS[0]:-0}
log "应用退出，代码: ${EXIT_CODE}"
exit ${EXIT_CODE}
```

6.4 Info.plist 配置

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
    "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>CFBundleName</key>
    <string>桌面宠物</string>
    <key>CFBundleDisplayName</key>
    <string>桌面宠物</string>
    <key>CFBundleIdentifier</key>
    <string>com.desktoppet.app</string>
```



```
<key>CFBundleVersion</key>
<string>7.0.0</string>
<key>CFBundleShortVersionString</key>
<string>7.0.0</string>
<key>CFBundleExecutable</key>
<string>桌面宠物</string>
<key>CFBundlePackageType</key>
<string>APPL</string>
<key>NSHighResolutionCapable</key>
<true/>
<key>LSUIElement</key>
<true/>
</dict>
</plist>
```

字段	说明
CFBundleIdentifier	应用唯一标识，建议反域名格式（com.company.product），升级时保持一致
CFBundleVersion	构建号，每次打包递增，用于系统判断版本新旧
CFBundleShortVersionString	用户可见的版本号（如 7.0.0）
LSUIElement = true	以「状态栏/后台应用」运行，不显示 Dock 图标，无主菜单栏（适合桌宠）
NSHighResolutionCapable = true	启用 Retina 高分屏支持，避免猫咪图像模糊

6.5 首次运行自动初始化

首次双击 .app 启动时，用户会看到以下流程（无 UI 弹窗，全部后台执行）：

阶段	耗时	说明
创建 venv	3~5 秒	python3 -m venv --copies 复制解释器
安装依赖	20~40 秒	pip install PyQt5 + requests（需联网）
检测 Qt 插件	< 1 秒	通过 QLibraryInfo 定位 plugins 目录

启动 main.py	1~3 秒	猫咪出现在屏幕右下角
------------	-------	------------

【提示】首次启动需联网安装 PyQt5（约 60MB）。若离线部署，可预打包 venv 并复制到目标机器的 ~/Library/Application Support/桌面宠物/ 下。

七、代码签名与分发

7.1 Ad-hoc 签名

build_app.sh 中使用 ad-hoc 签名（无开发者账号也能运行）：

```
# build_app.sh 第 243 行
codesign --force --deep --sign - "${APP_DIR}"

# --force 覆盖已有签名
# --deep 递归签名所有内嵌框架/二进制
# --sign - 使用 ad-hoc 身份（空签名），不校证书链
```

Ad-hoc 签名的局限性

- 用户首次打开时，macOS 会提示「无法验证开发者」，需手动到【系统设置 → 隐私与安全性 → 仍要打开】
- 不能分发到 App Store
- 不具备「公证（Notarization）」保护，可能被部分杀毒软件误报

（可选）开发者证书签名 + 公证

```
# 前提：已申请 Apple Developer ID 证书

# 1. 使用 Developer ID 签名
codesign --force --deep --sign "Developer ID Application: Your Name (TEAMID123)" --options runtime "dist/桌面宠物.app"

# 2. 打包成 DMG 用于分发
hdiutil create -volname "桌面宠物" -srcfolder "dist/桌面宠物.app" -ov -format UDZO "桌面宠物-v7.0.0.dmg"

# 3. 提交公证（需配置 app-specific password）
xcrun notarytool submit "桌面宠物-v7.0.0.dmg" --apple-id "your@apple-id.com" --team-id "TEAMID123" --password "app-specific-password" --wait
```

```
# 4. 公证通过后, 附加 ticket
xcrun stapler staple "桌面宠物-v7.0.0.dmg"
```

7.2 分发注意事项

分发渠道	单文件限制	建议
GitHub Releases	2GB	推荐, 配合 DMG 分发
百度网盘/阿里云盘	无限制	适合国内用户
网站直链下载	取决于服务器	需配置 Content-Type: application/octet-stream
QQ/微信传文件	~4GB	注意不要被识别为「风险文件」改名发送

八、常见问题与排查

8.1 启动日志定位

```
# 日志文件位置
cat ~/Library/Application\ Support/桌面宠物/launch.log

# 实时查看 (启动 .app 时)
tail -f ~/Library/Application\ Support/桌面宠物/launch.log

# 典型日志结构
[2026-08-12 10:00:00] ===== 桌面宠物启动 =====
[2026-08-12 10:00:00] RES_DIR: /Applications/桌面宠物.app/Contents/Resources
[2026-08-12 10:00:00] SUPPORT_DIR: /Users/suyue/Library/Application Support/桌面宠物
[2026-08-12 10:00:00] VENV_DIR: /Users/suyue/Library/Application Support/桌面宠物/venv
[2026-08-12 10:00:00] 首次启动, 使用系统 Python: /usr/local/bin/python3.11
[2026-08-12 10:00:01] 正在创建虚拟环境...
[2026-08-12 10:00:25] 正在安装依赖...
[2026-08-12 10:00:55] QT_PLUGIN_PATH: /Users/.../site-packages/PyQt5/Qt5/plugins
[2026-08-12 10:00:55] 启动 main.py...
[2026-08-12 10:00:56] [app] 视频播放器初始化成功
```

8.2 Qt 插件错误修复

错误: qt.qpa.plugin Could not find the Qt platform plugin "cocoa"

【注意】这是 PyQt5 打包部署最常见的错误。请按以下步骤排查:

1. 检查 launch.log 中是否有 QT_PLUGIN_PATH 输出, 为空则动态检测失败
2. 手动验证 PyQt5 安装完整性:

手动验证命令 (在 venv 中执行)

```
~/venv/bin/python -c "  
from PyQt5.QtCore import QLibraryInfo  
print('PluginsPath:', QLibraryInfo.location(QLibraryInfo.PluginsPath))  
import os  
p = QLibraryInfo.location(QLibraryInfo.PluginsPath)  
print('Exists:', os.path.isdir(p))  
print('Contents:', os.listdir(p) if os.path.isdir(p) else 'N/A')  
"
```

3. 若 PluginsPath 目录不存在, 重新安装 PyQt5:

```
pip uninstall -y PyQt5 PyQt5-Qt5 PyQt5-sip  
pip install --no-cache-dir PyQt5
```

8.3 视频无法播放排查

黑块/无画面: 视频编码不是 H.264, 用 VideoTranscoder.check_codec() 检查后转码

卡顿/掉帧: 视频分辨率过高, 转码时限制为 200×200 或 300×300

只有声音无图像: 检查 QMediaPlayer 是否 setMuted(False); 桌宠不需要音频, 保持 setMuted(True)

循环动作不循环: 确认 StoppedState 回调中正确调用 setPosition(0) + play() (顺序不可颠倒)

九、附录: 完整命令速查表

操作	命令
安装 FFmpeg	brew install ffmpeg
创建 venv	python3 -m venv venv && source venv/bin/activate
安装依赖	pip install -r requirements.txt

安装转码依赖	<code>pip install imageio imageio-ffmpeg pillow numpy</code>
MP4 → GIF 转码	<code>python convert_gifs.py</code>
GIF 优化压缩	<code>python optimize_gifs.py</code>
检测视频素材	<code>python check_videos.py</code>
开发模式运行	<code>bash run.sh</code> 或 <code>python main.py</code>
打包 .app	<code>bash build_app.sh</code>
查看启动日志	<code>tail -f ~/Library/Application\ Support/桌面宠物/launch.log</code>
Ad-hoc 签名	<code>codesign --force --deep --sign - dist/桌面宠物.app</code>